Lab Ten – Point Estimation

Tips

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with "**Solution**:".
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use "enter" to make your code readable, so it doesn't run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you're submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Don't forget that you can copy-and-paste code when you're doing something similar as we did in class!
- Make sure to plot all computed probabilities. This is good `ggplot` practice AND will help make sure you don't make a mistake when computing the probability.

1 Lab 8 Notes

1.1 Altham's Multiplicative Binomial Distribution

The PMF for Altham's Multiplicative Binomial Distribution is

$$f_X(x) = \frac{1}{c}\binom{n}{x}p^x(1-p)^{n-x}\theta^{x(n-x)}I(x \in \{0,1,...,n\})$$

where

$$c = \sum_{i=0}^{n}\binom{n}{i}p^i(1-p)^{n-i}\theta^{i(n-i)}.$$

Just so we start on the same page, I have included my `R` function for computing probability using this distribution.

```r
daltbinom <- function(x, size, prob, theta){
  normalizing.constant <- 0
  for(j in 0:size){
    normalizing.constant <- normalizing.constant +
      choose(size, j) * prob^j * (1-prob)^(size-j) * theta^(j*(size-j))
  }
  # Probability Mass at x=x
  (1/normalizing.constant) *
    choose(size, x) * prob^x * (1-prob)^(size-x) * theta^(x*(size-x)) *
      (x>=0)*(x<=size)
}
```

2 Altham's Multiplicative Beta Binomial Distribution

The PMF for Altham's Multiplicative Beta Binomial Distribution is

$$f_X(x) = \left[\int_0^1 \left(\frac{1}{c}\binom{n}{x}p^x(1-p)^{n-x}\theta^{x(n-x)}\right)\left(\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)}p^{\alpha-1}(1-p)^{\beta-1}\right)dp\right]I(x \in \{0,1,...,n\})$$

Just so we start on the same page, I have included my `R` function for computing probability using this distribution.

```r
daltbbinom.single <- function(x, size, alpha, beta, theta){
    integrand <- function(p) {
    sapply(p, # integrate will pass in a vector, so we use sapply to apply the following
           function(p_val){
             # f(x|p) * f(p|alpha, beta)
             daltbinom(x, size, p_val, theta) * dbeta(p_val, alpha, beta)
           }
    )
  }
  # Probability Mass at x=x
  # integrate(integrand, lower = 0, upper = 1)$value * (x>=0)*(x<=size)
  # I shrink the bounds below to avoid log(0)=-infinty
  # I use try() to catch when there is an error and avoid crashing
  res <- try(integrate(integrand, lower = 1e-7, upper = 1 - 1e-7)$value *
                 (x>=0)*(x<=size),
             silent = TRUE)
  # If there is an error, return a near-zero probability
  if(inherits(res, "try-error")) return(0.1^6)

  res
}
# Write a function that works on a vector
daltbbinom <- function(x, size, alpha, beta, theta){
  sapply(X=x, FUN=daltbbinom.single, size=size, alpha=alpha, beta=beta, theta=theta)
}
```

3 Question 1

In Lab 8, we use Altham's Multiplicative Binomial Distribution. Your job was to interpret the impact of θ in that equation. In this lab, we will model the number of legs won in a best of eleven (first to six) match.

To help us think about the interpretation, something that was challenging before, let's try something a little more manageable.

3.1 Part a

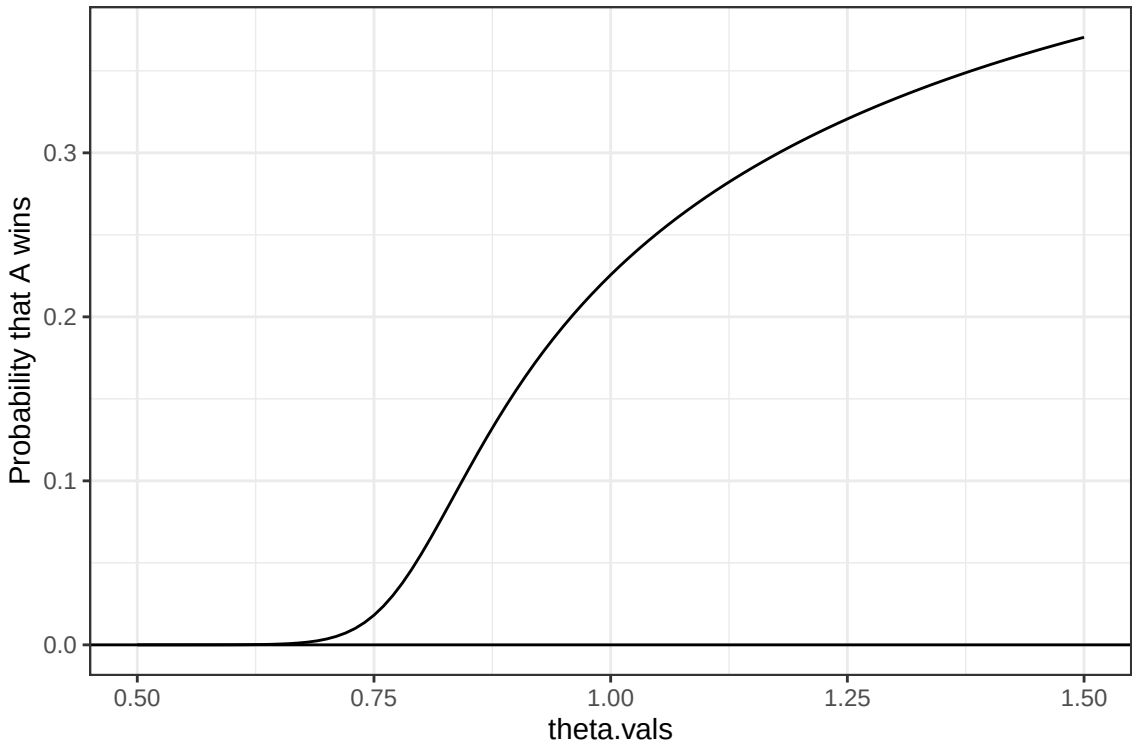
Suppose a player has a 50% chance of winning a specific leg, that is let $p = 0.50$. Let's look at the probability they win ($x = 6$) legs in the match. Compute this probability over a sequence from $\theta = 0.5$ to $\theta = 1.5$ and plot it using a line where the x-axis is θ and the y-axis is $P(X = 6)$.

Solution:

Created a sequence from 0.5 to 1.5 and created a tibble with the probabilities of the theta values in that range then plotted them using `geom_line()`

```r
theta.vals<- seq(0.5, 1.5, by=0.01) # theta vals from 0.5 to 1.5

# tibble
gg.dat<- tibble(theta.vals) |>
  mutate(PMF=daltbinom(size=11, prob=0.50, x=6, theta=theta.vals))

# plot
PMF.plot<- ggplot(data=gg.dat)+
 geom_line(aes(x=theta.vals,y=PMF))+
 geom_hline(yintercept=0)+
 theme_bw()+
 ylab("Probability that A wins")

PMF.plot
```

Figure 1: Probability that Player A wins as a function of under Altham's Multiplicative Binomial Distribution (n = 11, p = 0.5).



3.2 Part b

Now, plot Altham's Multiplicative Binomial Distribution under the same parameters with $\theta = 0.50$, $\theta = 1$, and $\theta = 1.5$. Use `ncol=1` in `facet_wrap()` to plot the PMFs in a single column for comparing

Solution

Plotted Altham's Multiplicative binom PMF with different theta values in a single column using `facet_wrap`

```
# tibble
gg.dat<- tibble(x=0:11) |>
  mutate(PMF05=daltbinom(x, size=11, prob=0.50, theta=0.50),
         PMF1=daltbinom(x, size=11, prob=0.50, theta=1.0),
         PMF15= daltbinom(x, size=11, prob=0.50, theta=1.5)
        )

(gg.dat)
```

```
# A tibble: 12 x 4
       x       PMF05     PMF1         PMF15
   <int>       <dbl>    <dbl>         <dbl>
 1     0 0.495       0.000488 0.00000000418
 2     1 0.00531     0.00537  0.00000265
 3     2 0.000104    0.0269   0.000340
 4     3 0.00000486  0.0806   0.0116
 5     4 0.000000608 0.161    0.118
 6     5 0.000000213 0.226    0.370
 7     6 0.000000213 0.226    0.370
 8     7 0.000000608 0.161    0.118
 9     8 0.00000486  0.0806   0.0116
10     9 0.000104    0.0269   0.000340
11    10 0.00531     0.00537  0.00000265
12    11 0.495       0.000488 0.00000000418
```

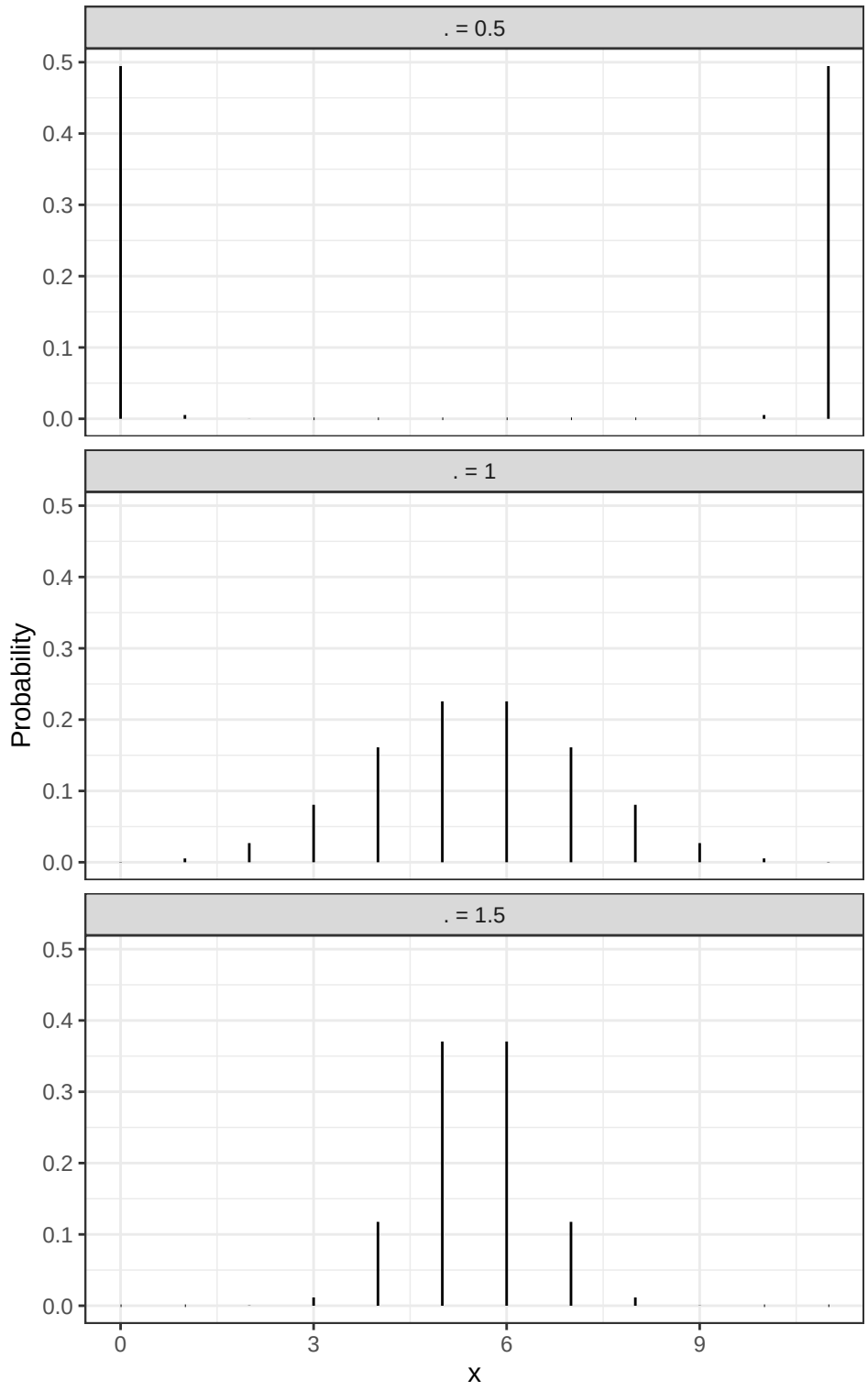
```
# pivot long
gg.long<- gg.dat |>
    pivot_longer(
    cols=starts_with("PMF"),
    names_to="theta",
    values_to="PMF"
  ) |>
```

```
  mutate(theta= factor(theta, levels= c("PMF05", "PMF1", "PMF15"), labels= c(" = 0.5", " = 1", " = 1.5")))
(gg.long)
```

```
# A tibble: 36 x 3
       x theta             PMF
   <int> <fct>           <dbl>
 1     0   = 0.5 0.495
 2     0   = 1   0.000488
 3     0   = 1.5 0.00000000418
 4     1   = 0.5 0.00531
 5     1   = 1   0.00537
 6     1   = 1.5 0.00000265
 7     2   = 0.5 0.000104
 8     2   = 1   0.0269
 9     2   = 1.5 0.000340
10     3   = 0.5 0.00000486
# i 26 more rows
```

```
# plot
PMF.plot<-ggplot(gg.long) +
  geom_linerange(aes(x=x, ymin=0, ymax=PMF)) +
  facet_wrap(~theta,ncol=1) +
  theme_bw() +
  xlab("x") +
  ylab("Probability")

PMF.plot
```

Figure 2: PMFs of Altham's Multiplicative Binomial Distribution for = 0.5, 1, and 1.5 (n = 11, p = 0.5), shown in a single column for comparison.



3.3 Part c

What is your best assessment of what θ is doing in this setting. Consider when a 6-0 blowout is most likely, and when a 5-6 nail-biter is most likely. Note here we should treat the match as a fixed $n = 11$ as we did for $n = 3$ in Lab 8.

Note: In lab 8, we saw that small θ meant wins came in bunches (clumped together), so there were more 2-0 wins in a best of three series.

Solution

The theta controls how the wins are spread out during the 11 legs. When theta<1 (0.5), wins happen in bunches so extreme results like 6-0 are more likely. When theta=1, the results are more evenly spread, and when theta>1 (1.5), the outcomes are more balanced and cluster around the middle, so close matches like 6–5 become more likely

4 Question 2

4.1 Part a

Altham's Multiplicative Binomial Distribution does not typically simplify to the clean $E(X) = np$ seen in the Binomial distribution, unless $\theta = 1$. Write out the expression that would need to be solved to find $E(X^k)$, and then write a function that computes it given the proposed parameter values p (`prob`) and θ (`theta`).

Solution

Created a function `compute.altbinom` which sums all values from 0 to size for `x^k * daltbinom()`

```
# size: scriptsize
compute.altbinom<-function(size, prob, theta, k){
  x<- 0:size
  form<-(x^k)*daltbinom(x, size=size, prob=prob, theta=theta)
  sum(form)
}
```

4.2 Part b

Describe how you can use your answer to part a to find Method of Moments estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution

To find the Method of Moments estimates for `p` and `theta` in Altham's Multiplicative Binomial Distribution, I match the theoretical moments of the model to the corresponding sample moments calculated from the data. Using the function `compute.altbinom`, we can compute the probabilities implied by the model and use them to obtain the model's mean and variance as functions of `p` and `theta`. From the data, we calculate the sample mean and sample variance, and then set these equal to the model's mean and variance to form a system of two equations. Solving these equations using a `nleqslv()` computationally in `R` gives the Method of Moments estimates for the parameters.

4.3 Part c

Write a function that computes the log-likelihood for Altham's Multiplicative Binomial Distribution, given data (`x`) and proposed parameter values p (`prob`) and θ (`theta`).

Solution

Created a function `loglik.altbinom` to find the log-likelihood for Altham's Multiplicative Binomial Distribution

```
# size: scriptsize

# function that finds the log likelihood
loglik.altbinom <- function(x, size, prob, theta){
  sum(log(daltbinom(x, size=size, prob=prob, theta=theta)))
}
```

4.4 Part d

Describe how you can use your answer to part c to find Maximum Likelihood estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution

I would use the function from part(c) to compute the log-likelihood of the observed data for different values of `p` and `theta`. I would then use an optimization approach computationally with `optim()` to identify the values that maximize the log-likelihood. The values of `p` and `theta` that achieve this maximum are taken as the Maximum Likelihood Estimates, since they correspond to the parameter combination under which the observed data are most probable.

5 Lab 10

Peter "Snakebite" Wright is a two-time Professional Darts Corporation World Champion (2020, 2022) and a former World No. 1. He is one of the sport's most decorated players, winning nearly 50 PDC titles including the World Matchplay, UK Open, and multiple European Championships.

In 2024, he made The Premier League – a league involving the eight best players that runs every Thursday night from February to May. In this season, he won two of eighteen matches, securing only four points. The spread in legs won was -43, meaning he lost 43 more legs than he won. This performance was rather quite bad. He was soundly trounced in almost all his match ups.

The 2024 standings are below:

Rank	Player
1	Luke Littler
2	Luke Humphries
3	Michael van Gerwen
4	Michael Smith
5	Nathan Aspinall
6	Rob Cross
7	Gerwyn Price
8	Peter Wright

5.1 Summarize the 2024 Data

In `pdc2024.csv`, I have provided the data for the 2024 Premier League season. There are a few data cleaning steps to consider.

- Remove any matches with no `Score` (e.g., a bye or walkover)
- Nights 1 through 16 are best of 11 (first to 6), but the playoffs are extended. For consistency, keep nights 1 through 16 only.
- Make two columns `WinnerLegs` and `LoserLegs` that keep track of the number of legs each player has won

Summarize the data. What do the data tell us about the breakdown of the results?

Solution

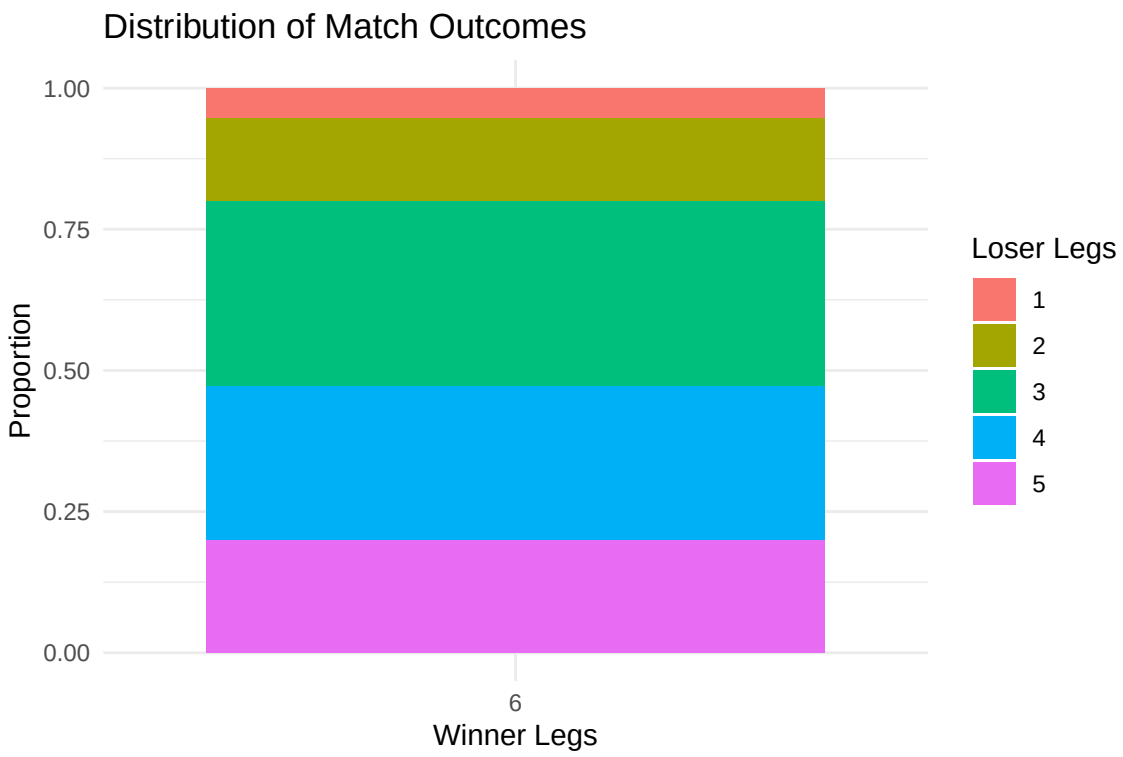
Read the csv file and cleaned the data by removing matches with missing scores, w/o, and keeping only nights 1–16 then separated the Score column into WinnerLegs and LoserLegs. Summarized the resulting data.

```
# read csv file
prem.dat<- read_csv("pdc2024.csv")

# clean dat
prem.dat.cleaned <-prem.dat |>
  filter(!is.na(Score)) |> # only none null values
  filter(Score != "w/o") |> # remove values shown as w/o
  filter(Night <= 16) |> # nights 1-16
  separate(Score, into=c("WinnerLegs","LoserLegs"), sep="-") # seperated by -

# summary
summary.dat <- prem.dat.cleaned |>
  count(WinnerLegs, LoserLegs) |> # frequency
  mutate(Proportion=n/sum(n)) # proportion

(summary.dat)
```

```
# A tibble: 5 x 4
  WinnerLegs LoserLegs     n Proportion
  <chr>      <chr>     <int>      <dbl>
1 6          1             3     0.0545
2 6          2             8     0.145
3 6          3            18     0.327
4 6          4            15     0.273
5 6          5            11     0.2
```

```r
# plot summary
ggplot(summary.dat, aes(x = factor(WinnerLegs), y = Proportion, fill = factor(LoserLegs))) +
  geom_bar(stat = "identity") +
  labs(
    title = "Distribution of Match Outcomes",
    x = "Winner Legs",
    y = "Proportion",
    fill = "Loser Legs"
  ) +
  theme_minimal()
```

Figure 3: Distribution of match outcomes by winner and loser legs.



5.2 Altham's Multiplicative Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maximium likelihood estimates for p and θ.
Note: The "for each" here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLE for the player using this data. Ensure to report p and θ for each player in a nicely formatted table.
  Note: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space. For example, here, you might consider `p <- 1/(1+exp(-par[1]))` and `theta <- exp(par[2])`.
- Make comments about the MLEs in the context of the standings.

Solution

Created a vector of all players and used looped through it to compute the MLE for each player. For each player, I filtered the matches they played, made a vector of number of legs won, and calculated total legs in each match then

used optim() with transformed parameters to estimate `p` and `theta`.

```
players<- unique(c(prem.dat.cleaned$Winner, prem.dat.cleaned$Loser))

# variables to store values
p_estims<- rep(NA, length(players))
theta_estims<- rep(NA, length(players))

# Define negative log-likelihood function for Altham's multiplicative binom
l.likely<- function(par, x, size, neg = FALSE){

  # transformations for parameters
  p<- 1 / (1 + exp(-par[1]))
  theta<- exp(par[2])

  # Compute log-likelihood using the Altham binom PMF
  ll<- sum(log(daltbinom(x=x, size=size, prob=p, theta=theta)))

  # Return negative log-likelihood when neg=TRUE in optim
  ifelse(neg, -ll, ll)
}

# loop through each player
for(i in 1:length(players)){
  player<-players[i]

  # Matches
  matches<-prem.dat.cleaned |>
    filter(Winner==player|Loser==player)

  # legs won by player
  legs_won<-as.numeric(if_else(matches$Winner==player, matches$WinnerLegs, matches$LoserLegs))

  # total legs by player
  total_legs<- as.numeric(matches$WinnerLegs) + as.numeric(matches$LoserLegs)

  # find MLE with optim
  res<- optim(par=c(0,0), fn=l.likely, x=legs_won, size=total_legs, neg=TRUE)
  # Convert and store in variable
  p_estims[i]<- 1/(1 + exp(-res$par[1]))
  theta_estims[i]<- exp(res$par[2])
}

# store values(3dp) in tibble
results <- tibble(
  Player=players,
  p=round(p_estims,3),
  theta=round(theta_estims,3)
)

(results)
```

```
# A tibble: 8 x 3
  Player                p theta
  <chr>             <dbl> <dbl>
1 Rob Cross         0.437  1.04
2 Gerwyn Price      0.474  1.03
3 Michael Smith     0.543  1.08
4 Luke Littler      0.636  1.12
5 Nathan Aspinall   0.506  1.10
6 Luke Humphries    0.504  1.03
7 Michael van Gerwen 0.484  1.07
8 Peter Wright      0.005  5.09
```

Comments

The players with higher estimated `p` values generally seem to match the stronger players in the standings, since a larger `p` means they were more likely to win legs overall while players with lower `p` values were usually the ones lower down the table, since they won fewer legs on average. The `theta` values control the pattern and how spread out the wins were. Players with smaller `theta` values tended to have results that were more uneven, while players with larger `theta` had matches that were more balanced.

5.3 Altham's Multiplicative Beta Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maxmium likelihood estimates for p and θ.
Note: The "for each" here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.

- Find the MLEs for the player using this data. Ensure to report α, β, and θ for each player in a nicely formatted table.
  Note 1: Due to the computational complexity of `integrate()`, you should compute the logged probability for `0:6` once and add them according to the data in each player's vector. **Note 2**: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space.
- Plot the underlying Beta(α, β) distribution for each player.
  Note: Add the mean and variance of the Beta to the table of MLEs. Recall

$$E(X) = \frac{\alpha}{\alpha + \beta}$$

$$sd(X) = \sqrt{\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}}$$

- Make comments about the MLEs in the context of the standings.
  Note: The players alternate who throws first in a leg has the mathematical "head start" to reach a finish before their opponent can take another turn. Players alternate who throws first with each leg.

Solution

Used a loop to estimate alpha, beta, and theta values for each player using Altham's Multiplicative Beta-Binom. I created a vector of legs won across matches for each player and computed negative log likelihood. I also computed the mean and standard deviation of the corresponding Beta distribution and summarized the results in a table for all players.

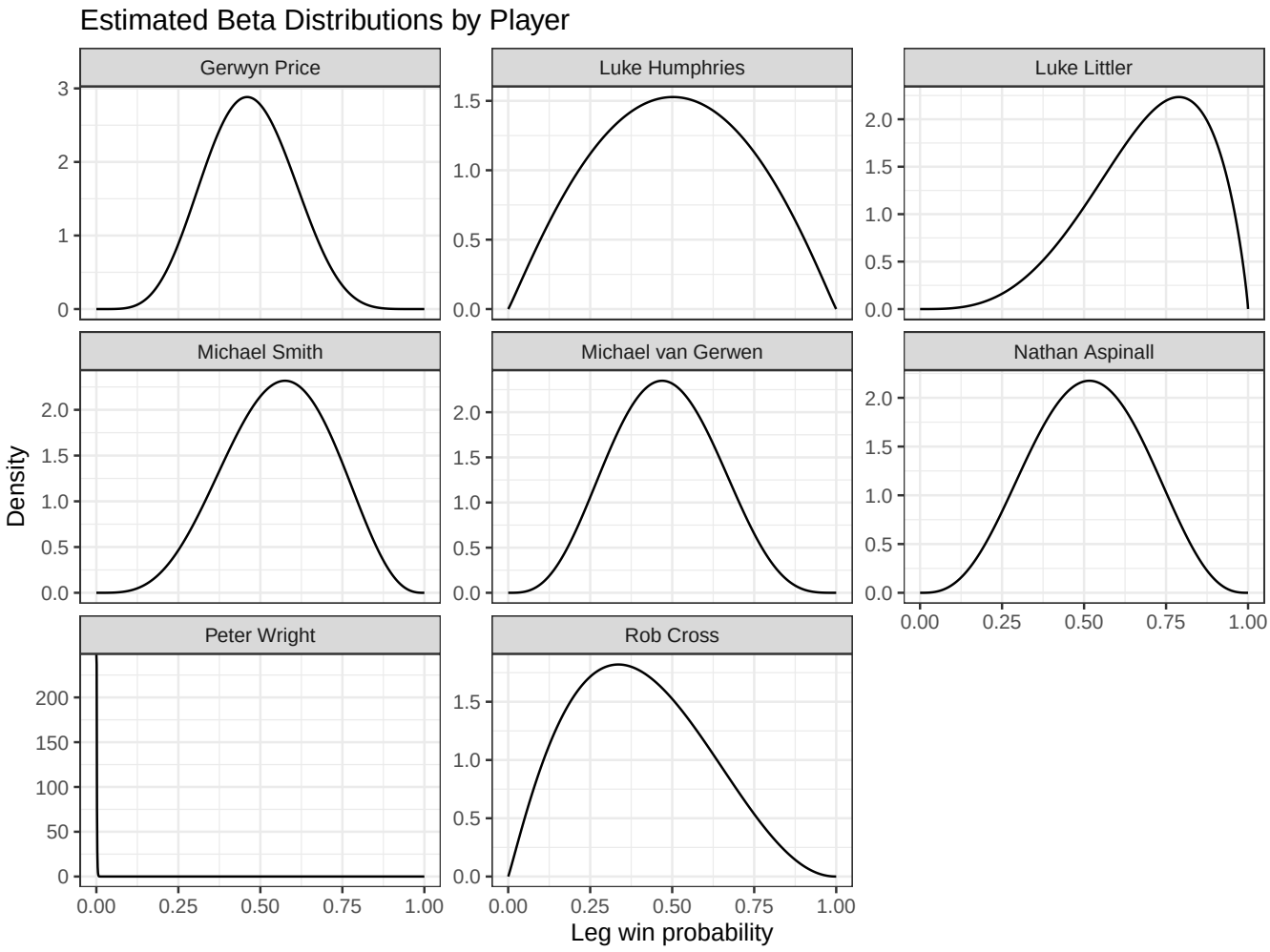
```
# variables to store values
alpha_estims<- rep(NA, length(players))
beta_estims<- rep(NA, length(players))
theta_estims<- rep(NA, length(players))
beta_mean<- rep(NA, length(players))
beta_sd<- rep(NA, length(players))

# negative log-likelihood for Altham multiplicative beta-binom
l.likely.bb <- function(par, x, size, neg = FALSE){
  # transformations
  alpha<- exp(par[1])
  beta<- exp(par[2])
  theta<- exp(par[3])

  # compute log-likelihood match by match (size can vary per match)
  ll<- sum(mapply(function(xi, si){
    # compute probs for 0:si once per match
    probs <- daltbbinom(0:si, size=si, alpha=alpha, beta=beta, theta=theta)
    log(probs[xi + 1])
  }, x, size))

  ifelse(neg, -ll, ll)
}

# loop through players
for(i in 1:length(players)){
  player<- players[i]

  # matches
  matches<- prem.dat.cleaned |>
    filter(Winner == player|Loser == player)

  # legs won and total legs per match
  legs_won<- as.numeric(if_else(matches$Winner == player,
                                matches$WinnerLegs, matches$LoserLegs))
  total_legs<- as.numeric(matches$WinnerLegs) + as.numeric(matches$LoserLegs)

  # MLE with optim
  res<- optim(par=c(0, 0, 0), fn=l.likely.bb,
              x=legs_won, size=total_legs, neg=TRUE)

  # convert and store
  alpha_estims[i]<- exp(res$par[1])
  beta_estims[i]<- exp(res$par[2])
  theta_estims[i]<- exp(res$par[3])

  # beta mean and sd
  beta_mean[i]<- alpha_estims[i]/(alpha_estims[i]+beta_estims[i])
  beta_sd[i]<- sqrt((alpha_estims[i] * beta_estims[i])/((alpha_estims[i] + beta_estims[i])^2*(alpha_estims[i] + beta_estims[i]+1)))
}

# results table
results<- tibble(
```

```
  Player = players,
  alpha = round(alpha_estims, 3),
  beta = round(beta_estims,  3),
  theta = round(theta_estims, 3),
  Mean = round(beta_mean,    3),
  SD = round(beta_sd,      3)
)
results
```

```
# A tibble: 8 x 6
  Player             alpha   beta theta  Mean    SD
  <chr>              <dbl>  <dbl> <dbl> <dbl> <dbl>
1 Rob Cross           2.08   3.14  1.31 0.399 0.196
2 Gerwyn Price        6.27   7.19  1.15 0.466 0.131
3 Michael Smith       4.88   3.87  1.27 0.558 0.159
4 Luke Littler        4.24   1.87  1.49 0.694 0.173
5 Nathan Aspinall     4.05   3.86  1.31 0.512 0.168
6 Luke Humphries      2.07   2.06  1.37 0.501 0.221
7 Michael van Gerwen  4.34   4.78  1.25 0.476 0.157
8 Peter Wright        0.53 817.   15.4  0.001 0.001
```

```
# beta distribution plot
p_vals<- seq(0, 1, length.out = 1000)

beta.plot.dat<- tibble()

for(i in 1:length(players)){
  beta.plot.dat<- bind_rows(beta.plot.dat, tibble(
    Player = players[i],
    p = p_vals,
    density = dbeta(p_vals, alpha_estims[i], beta_estims[i])
  ))
}

# plot
ggplot(beta.plot.dat, aes(x = p, y = density)) +
  geom_line() +
  facet_wrap(~ Player, scales = "free_y") +
  theme_bw() +
  labs(
    title = "Estimated Beta Distributions by Player",
    x     = "Leg win probability",
    y     = "Density"
  )
```

Figure 4: Estimated Beta distributions for each player based on MLEs.



5.4 Executive Summary

Summarize the work you've done here to provide a brief (200-300 word) data-driven summary of the 2024 Premier League season using your work in Lab 10. Your summary should be professional, clear, and all claims should be corroborated with statistical support.

Solution:

I read in the CSV file and cleaned the data by removing matches with missing scores, excluding those marked as "w/o", and restricting the dataset to nights 1–16. I then separated the Score column into WinnerLegs and LoserLegs and created a summary table of frequencies and proportions for each score combination.

The summary table and corresponding bar plot show that most matches are relatively competitive, with outcomes concentrated around common scores like 6–4 and 6–5, while more extreme results like 6–0 or 6–1 occur much less frequently. This indicates that, although some matches are one-sided, the majority of games are fairly balanced between players. The data suggest that close matches are the most typical outcome in the 2024 Premier League season, with only occasional blowout results